

CETEC UBATIC 2026-27

Grupo 2: Qdrant + CLIP/CONCH + LangGraph/ADK

Informe Técnico - Sprint 1

Ingestión, Chunking Multimodal y Evaluación de Embeddings

Mateo Gonzalez Pautaso - Alen Calandria

Abril 2026

1. Contexto y Objetivo del Sprint 1

El proyecto CETEC UBATIC 2026-27 consiste en construir un sistema RAG (Retrieval-Augmented Generation) multimodal para consulta de un manual de histología médica. El sistema debe poder responder preguntas sobre tejidos, células y estructuras histológicas tanto a partir de texto como de imágenes microscópicas.

El Grupo 2 trabaja con la siguiente arquitectura tecnológica:

Componente	Tecnología	Rol
Base de datos vectorial	Qdrant	Almacenamiento y búsqueda por similitud coseno
Modelo de embeddings	CLIP / CONCH	Generación de vectores de texto e imagen
Orquestación agente	LangGraph / ADK	Grafo de estados del pipeline RAG
LLM de generación	Gemini 2.5 Flash	Producción de respuestas multimodales
LLM de evaluación	Groq Llama 3.3 70B	Métricas RAGAS

Los objetivos del Sprint 1 fueron:

- Determinar el tamaño de chunk óptimo para que el modelo de embeddings mantenga el contexto semántico de una microfotografía junto a su descripción.
- Evaluar si CLIP genérico es suficiente para retrieval visual en histología H&E, o si se requiere un modelo especializado.
- Construir el pipeline de evaluación con métricas Recall@K y RAGAS.

2. Arquitectura del Sistema RAG Multimodal v3.2

2.1 Visión General

El sistema está organizado en cuatro capas que interactúan de forma asincrónica a través de un grafo LangGraph:

Capa	Componente	Funcion
Presentación	modo_interactivo()	Bucle de chat en terminal. Gestiona input de texto e imagen por turno.
Orquestación	LangGraph StateGraph	Define y ejecuta el grafo de nodos que procesa cada consulta.
Recuperación	Qdrant + CLIP/CONCH	Busca fragmentos de texto e imágenes usando similitud coseno.

Capa	Componente	Funcion
Generación	Gemini 2.5 Flash	Produce la respuesta integrando contexto RAG, análisis visual e historial.

2.2 Grafo de Nodos LangGraph

Cada consulta del usuario recorre los siguientes nodos en orden:

#	Nodo	Descripción
1	inicializar	Inicializa todos los campos del estado: contexto de memoria, temario, flags.
2	procesar_imagen	Gestiona la imagen: nueva, reutiliza activa, o modo texto puro. Genera embedding CLIP y análisis visual con Gemini.
3	clasificar	Doble clasificación: tipo de consulta vía LLM y validación de dominio vía ClasificadorSemantico (CLIP + LLM).
4a	fuera_temario	Responde que la consulta no pertenece al dominio histológico.
4b	generar_consulta	Genera queries cortas optimizadas para Qdrant.
5	buscar_qdrant	Búsqueda vectorial: una por texto y otra por embedding de imagen. Combina y deduplica.
6	filtrar_contexto	Filtra resultados con similitud ≥ 0.22 . Construye contexto de documentos.
7	analisis_comparativo	Compara imagen del usuario vs referencias recuperadas usando Gemini multimodal.
8	generar_respuesta	Produce la respuesta final integrando todos los elementos.
9	finalizar	Guarda la interacción en SemanticMemory y registra la trayectoria en JSON.

2.3 Configuración de Qdrant

Parámetro	Valor
Nombre de la colección	histologia_multimodal
Métrica de distancia	COSINE (similitud coseno)
Dimensión del vector (CLIP)	512 dimensiones
Dimensión del vector (CONCH)	768 dimensiones
Umbral de aceptación	0.22 (SIMILARITY_THRESHOLD)
Top-K por búsqueda	10 por modalidad (texto + imagen)
Resultados combinados	Hasta 15 (de duplicados)

2.4 Memoria Semántica Persistente

El sistema implementa SemanticMemory con las siguientes capacidades:

- Persistencia de imagen entre turnos del chat: la imagen activa se recuerda hasta que el usuario la cambie explícitamente con el comando 'nueva imagen'.
- Resumen automático del historial: cuando supera 6 interacciones, el LLM resume el contexto previo para ahorrar tokens.
- Persistencia de memoria conversacional incluso cuando el RAG no encuentra contexto suficiente.

2.5 Clasificador Semántico

Reemplaza la verificación por keywords con un clasificador que combina:

- Similitud coseno entre el embedding CLIP de la consulta y 12 anclas semánticas predefinidas del dominio histológico. Umbral: 0.18.
- Razonamiento LLM como árbitro en casos ambiguos. Si hay imagen activa, el umbral efectivo se reduce al 60% para aceptar preguntas implícitas.

3. Diagnóstico del Sistema: Problema Inicial Identificado

Al analizar la primera respuesta generada por el sistema para una consulta sobre Cartílago Hialino, se identificaron los siguientes problemas:

3.1 Ausencia de grounding visual

La respuesta fue correcta en contenido histológico textual pero completamente ciega a las imágenes del manual. El sistema admite directamente que no había imagen de entrada y operó en modo texto puro, ignorando las microfotografías indexadas.

Esto reveló el problema estructural central: imagen y texto estaban indexados como puntos separados y sin relación entre sí en Qdrant. El chunk de texto descriptivo y la microfotografía de la misma lámina eran dos vectores distintos sin vínculo.

3.2 Desacoplamiento imagen-texto en la indexación

El pipeline original de indexación tenía este comportamiento:

- Texto: el PDF se dividía en chunks de ~500 caracteres sin saber que había una imagen asociada.
- Imagen: cada página se renderizaba e indexaba sola, con solo el OCR como contexto.

El resultado es que el chunk de texto con descripción de 'condrocitos en lagunas, matriz territorial...' y la imagen de cartílago hialino de la Imagen 11.2 estaban en Qdrant como dos puntos separados. La búsqueda por texto encontraba el fragmento textual pero no la imagen; la búsqueda por imagen encontraba otras imágenes pero sin su descripción histológica.

3.3 Solución propuesta: chunks multimodales por página

La solución correcta es generar una unidad atómica imagen+texto+metadatos por pagina, donde cada punto en Qdrant contenga:

- El embedding visual de la imagen de la pagina (CLIP/CONCH)
- El texto OCR enriquecido con metadatos de las tablas (tejido, variedad, estructura señalada)
- El path de la imagen renderizada
- Metadatos estructurados del manual (laminilla, órgano, tejido, variedad)

4. Pipeline de Evaluación Implementado

4.1 Evaluacion_ChunkSize_Histologia.ipynb

Evalúa tres estrategias de chunking usando Recall@K y RAGAS con Groq como LLM evaluador.

Golden Set

Se construyó un golden set de 20 pares pregunta-respuesta extraídos directamente del manual arch2.pdf, cubriendo las cinco prácticas:

- Practica 11: Cartílago (hialino, elástico, fibroso, embrionario) - 8 preguntas
- Práctica 12: Tejido óseo - 4 preguntas
- Práctica 13: Tejido muscular y sarcómera - 3 preguntas
- Practica 14: Neurona - 2 preguntas
- Practica 15: Neuroglia - 3 preguntas

Configuraciones evaluadas

Colección	Estrategia	Descripción
histologia_eval_pagina	Página completa	Un chunk por página, texto enriquecido con metadatos de tablas
histologia_eval_500	Chunks de 500 chars	Chunks de 500 caracteres con overlap del 20%
histologia_eval_250	Chunks de 250 chars	Chunks de 250 caracteres con overlap del 20%

LLM Evaluador

Se eligió Groq con Llama 3.3 70B como LLM evaluador por las siguientes razones:

- Gemini Flash tenía un límite de 15 RPM en el tier gratuito que se agotaba rápidamente con RAGAS.
- Groq ofrece 500,000 tokens por minuto y 100,000 tokens por día en el tier gratuito.
- El 70B es necesario para las tareas de razonamiento estructurado que requiere RAGAS (el 8B produce outputs mal formateados).

Importante: los scores RAGAS obtenidos con Groq no son directamente comparables con los de otros grupos que usen Gemini. Para la comparativa del Sprint 6, coordinar con todos los grupos para usar el mismo modelo evaluador.

4.2 Test imagen->imagen

Se implementó un test de retrieval visual que evalúa si el sistema recupera la página correcta cuando se le pasa una imagen como query. Por cada una de las 35 páginas del manual:

- Se genera el embedding de la imagen con el modelo a evaluar.
- Se busca en Qdrant y se verifica si la página correcta aparece en los top-1, top-3 y top-5.
- Se diagnostica la causa de cada falla: score bajo (threshold/imagen pequeña) vs score ok pero página incorrecta (problema de chunk/modelo).

5. Resultados

5.1 Evaluación del tamaño de chunk (texto)

Configuración	Recall@1	Recall@3	Recall@5
Página completa	0.100	0.200	0.400
Chunks de 500 chars	0.200	0.400	0.450
Chunks de 250 chars	0.200	0.300	0.350

Conclusión: la configuración de 500 caracteres es la mejor para retrieval de texto, con Recall@3 = 0.40. Sin embargo, estos valores son bajos en términos absolutos, indicando que el problema de retrieval va más allá del tamaño de chunk.

5.2 Evaluación de retrieval imagen->imagen

Modelo	R@1	R@3	R@5	Observaciones
CLIP ViT-B/32	0.000	0.086	0.143	32 de 35 páginas fallan. Score ok pero página incorrecta.
CONCH ViT-B/16	1.000	1.000	1.000	35 de 35 páginas recuperadas correctamente.

Resultado crítico: CLIP genérico es inapropiado para retrieval visual en histología H&E. Las imágenes de cartílago hialino, cartílago elástico, tejido muscular y tejido nervioso se ven visualmente similares (manchas rosas con células pequeñas) para un modelo entrenado en internet general. CONCH, entrenado específicamente en patología, las distingue perfectamente.

5.3 Diagnóstico de fallas de CLIP

El análisis de las 32 páginas que fallaron con CLIP reveló un patrón consistente:

- Score siempre mayor a 0.22 (el threshold): no es un problema de threshold ni de imágenes pequeñas.
- Las paginas 23 (sarcomera, imagen en blanco y negro) y 10 (osteona, escala de grises) aparecen como falsos positivos en casi todos los errores.
- Causa raíz: CLIP no puede distinguir cartílago hialino de cartílago elástico porque ambos son manchas rosas con células similares a nivel de píxeles.

5.4 Métricas RAGAS

La evaluación RAGAS con Groq Llama 3.3 70B no pudo completarse exitosamente por dos razones:

- Límite diario de 100,000 tokens de Groq agotado al procesar las 20 preguntas x 3 configuraciones x 3 métricas = 180 llamadas al LLM.
- El modelo 8B produce outputs mal formateados (OutputParserException) que RAGAS no puede parsear correctamente.

Para el Sprint 6 se recomienda:

- Reducir el golden set a 7-10 preguntas para la evaluación RAGAS definitiva.
- Coordinar con todos los grupos para usar el mismo modelo evaluador.
- Considerar usar la API de Gemini con rate limiting manual en lugar de Groq.

6. CONCH: Modelo de Embeddings Especializado en Patología

6.1 Qué es CONCH

CONCH (Contrastive Learning from Captions in Histopathology) es un modelo de embeddings desarrollado por el laboratorio de IA de Harvard Medical School (MahmoodLab). A diferencia de CLIP, fue entrenado sobre millones de pares imagen-descripción de patología médica, lo que le permite:

- Distinguir subtipos de tejido histológico visualmente similares (cartílago hialino vs elástico).
- Entender la relación semántica entre microfotografías H&E y descripciones anatómicas.
- Generar embeddings en el mismo espacio vectorial para texto e imagen de dominio médico.

Atributo	CLIP ViT-B/32	CONCH ViT-B/16
Entrenamiento	Internet general	Patología + histología médica
Dimensión de embedding	512	768
Distingue H&E similares	No	Si
Disponibilidad	HuggingFace público	HuggingFace (requiere aceptar términos)
Recall@1 en histología	0.000	1.000

6.2 Proceso de carga del modelo

El repositorio MahmoodLab/conch contiene solo el checkpoint `pytorch_model.bin` sin `config.json` ni estructura de Transformers. El proceso de carga correcto es:

Paso 1: Obtener acceso y token

- Ir a huggingface.co/MahmoodLab/conch y aceptar el acuerdo de uso.
- Crear un token con permisos de lectura en huggingface.co/settings/tokens.
- Guardar el token como secreto `HF_TOKEN` en Google Colab.

Paso 2: Descargar y analizar el checkpoint

```
checkpoint_path = hf_hub_download(  
    repo_id='MahmoodLab/conch',  
    filename='pytorch_model.bin',  
    token=HF_TOKEN  
)  
checkpoint = torch.load(checkpoint_path, map_location='cpu')
```


Paso 3: Cargar encoder visual con timm

Las keys del encoder visual usan la nomenclatura trunk.* (timm-style), no compatible con open_clip ni con CLIPVisionModel. La solución es usar timm directamente:

```
conch_vit = timm.create_model(
    'vit_base_patch16_224',
    pretrained=False,
    num_classes=0 # devuelve features, no logits
)
visual_sd = {
    k.replace('visual.', ''): v
    for k, v in checkpoint.items()
    if k.startswith('visual.')
}
visual_sd.pop('proj_contrast', None)
missing, unexpected = conch_vit.load_state_dict(visual_sd, strict=False)
```

Resultado: 150 missing keys y 174 unexpected keys. Si bien no es una carga perfecta (CONCH tiene capas adicionales específicas), el modelo produce embeddings de 768 dimensiones que logran Recall@1 = 1.000 en el test imagen->imagen.

Paso 4: Función de embedding

```
conch_transform = transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

def embed_conch(pil_img):
    tensor = conch_transform(pil_img).unsqueeze(0).to(device)
    with torch.no_grad():
        emb = conch_vit(tensor).cpu().numpy().flatten()
    return (emb / np.linalg.norm(emb)).tolist()
```

6.3 Problemas encontrados durante la integración

Problema	Causa	Solución
Error 401 con <code>timm.create_model</code>	<code>timm</code> lee variable de entorno <code>HF_TOKEN</code> de Colab (token viejo) ignorando el <code>login()</code>	Eliminar <code>os.environ['HF_TOKEN']</code> antes del login y volver a setearlo después.
Error 404 con 'hf-hub:MahmoodLab/CONCH'	El repositorio es case-sensitive: <code>conch</code> (minúsculas)	Usar 'hf-hub:MahmoodLab/conch'
CONCH no tiene <code>config.json</code>	No es un modelo Transformers estándar, sino un checkpoint PyTorch custom	Cargar manualmente con <code>timm</code> y mapear las keys.
<code>open_clip</code> : 152 missing keys	Nomenclatura diferente: <code>trunk.*</code> en CONCH vs nombres propios de <code>open_clip</code>	Cambiar a <code>timm</code> que sí entiende la nomenclatura <code>trunk.*</code>
<code>timm</code> : 150 missing keys	CONCH tiene capas adicionales (<code>proj_contrast</code> , etc.) sin equivalente en ViT-B/16 estándar	Aceptado como limitación: el resultado sigue siendo <code>Recall@1=1.000</code>

7. Organización de Notebooks

7.1 CETEC_g2_v3_2.ipynb - Sistema Principal

El notebook principal fue refactorizado de una celda monolítica a 11 celdas separadas para facilitar la reanudación de sesiones en Colab:

Celda	Descripción	Frecuencia
1-3	<code>pip install</code> (base, ImageBind, adicionales)	Una vez por sesión
4	Credenciales y constantes globales	Una vez por sesión
5	Imports generales y utilidades	Una vez por sesión
6	Clases auxiliares (SemanticMemory, Clasificador, etc.)	Una vez por sesión
7	Clase principal AsistenteHistologiaMultimodal	Una vez por sesión
8	Inicializar componentes (carga CLIP + LLM)	Punto de reanudación si Colab desconecta
9	Procesar PDFs y extraer temario	Una vez por sesión
10	Indexar en Qdrant	Saltear si ya esta indexado
11	Consulta interactiva	Cada vez que se consulta

7.2 Evaluacion_ChunkSize_Histologia.ipynb - Evaluación

Bloque 1: Evaluación del tamaño de chunk (celdas 1-10)

- Instala dependencias, carga credenciales y CLIP.
- Inspecciona el PDF y genera 3 colecciones en Qdrant con chunks de distinto tamaño.
- Golden set de 20 preguntas extraídas del manual real.
- Recall@K sin LLM (celda 8, recomendada como primer paso).
- Métricas RAGAS completas con Groq (celda 9, consume cuota).
- Tabla comparativa final (celda 10).

Bloque 2: Diagnóstico imagen->imagen (celdas 11-14)

- Instalación de CONCH y carga del token HuggingFace (celda 11).
- Test imagen->imagen con CLIP (celda 12).
- Test imagen->imagen con CONCH: descarga checkpoint, carga con timm, indexa y evalúa (celda 13).
- Reporte comparativo CLIP vs CONCH con diagnóstico de causas (celda 14).

8. Próximos Pasos

8.1 Inmediato - Coordinación con otros grupos

Compartir el archivo reporte_diagnostico_imagenes.json con G1, G3 y el equipo de métricas para que evalúen si obtienen resultados similares con CLIP. Si todos tienen Recall@1 bajo con CLIP en histología H&E, la decisión de migrar a CONCH se toma a nivel de proyecto.

8.2 Sprint 2 - Integrar CONCH en el sistema principal

Los cambios necesarios en CETEC_g2_v3_2.ipynb para migrar de CLIP a CONCH son:

- Reemplazar `_init_clip()` por el pipeline de carga de CONCH (Pasos 1-4 de la celda 13 del notebook de evaluación).
- Actualizar la dimensión del vector en Qdrant de 512 a 768.
- Actualizar `_embed_imagen()` para usar `embed_conch()`.
- El `_embed_texto()` puede mantenerse con CLIP ya que CONCH no tiene encoder de texto accesible de forma simple.

8.3 Sprint 2 - Implementar chunking multimodal

Reemplazar el chunking actual por chunks que preserven la unidad semántica imagen-metadatos-texto:

- Usar pdfplumber para extraer texto y tablas por página.

- Enriquecer cada chunk con los metadatos de las tablas histológicas (tejido, variedad, estructura señalada).
- Indexar con named vectors duales en Qdrant: uno visual (CONCH) y uno textual (CLIP).

8.4 Sprint 2 - Ontología

El plan de trabajo exige la extracción inicial de la ontología jerarquica Órgano -> Tejido -> Célula para el Sprint 1. Esta ontología es necesaria para el Sprint 3 donde el agente la usará para filtrar el espacio de búsqueda antes de aplicar similitud coseno.

Estructura mínima a implementar:

- Sistema Musculoesqueletico > Cartilago > [Hialino, Elástico, Fibroso]
- Sistema Oseo > Hueso Compacto > [Osteona, Osteoblasto, Osteocito, Osteoclasto]
- Sistema Muscular > [Estriado Voluntario, Estriado Involuntario, Liso]
- Sistema Nervioso > [Neurona, Neuroglia]

9. Conclusiones del Sprint 1

Hallazgo	Impacto	Acción
CLIP tiene Recall@1=0.000 para retrieval visual en histología H&E	Alto: el sistema multimodal es inutil para busqueda por imagen con CLIP	Migrar a CONCH en Sprint 2
CONCH tiene Recall@1=1.000 con el mismo dataset	Alto: confirma que el problema es el modelo, no la arquitectura	Integrar CONCH en sistema principal
Chunk de 500 chars es mejor que pagina completa para texto	Medio: R@3 sube de 0.20 a 0.40	Implementar chunking multimodal por página en Sprint 2
Imagen y texto indexados por separado rompen el retrieval multimodal	Alto: causa principal de las fallas	Chunks con unidad semántica imagen+texto+metadatos
RAGAS requiere modelos grandes (70B+) para resultados confiables	Medio: limita la evaluación con tier gratuito de Groq	Reducir golden set y coordinar modelo evaluador entre grupos

El Sprint 1 confirma que la arquitectura del sistema es correcta pero requiere dos cambios fundamentales en la capa de inversión: reemplazar CLIP por CONCH y reformular el chunking para preservar la unidad semántica de cada lámina histológica. Con estos cambios, el sistema estará en condiciones de producir retrieval multimodal de alta precisión para el Sprint 2.

Apéndice: Datos del Proyecto

Campo	Valor
Proyecto	CETEC UBATIC 2026-27
Grupo	2
Integrantes	Mateo Gonzalez Pautaso - Alen Calandria
Stack tecnológico	Qdrant + CLIP/CONCH + LangGraph/ADK + Gemini 2.5 Flash
Manual utilizado	arch2.pdf (35 páginas, 25 laminas histologicas)
Prácticas cubiertas	Prácticas 11 a 15 (Cartílago, Hueso, Músculo, Neurona, Neuroglia)
Colecciones Qdrant creadas	histologia_multimodal, histologia_eval_pagina, histologia_eval_500, histologia_eval_250, histologia_conch_pagina
Sprint actual	Sprint 1 - Ingestion y Chunking Multimodal
Fecha del informe	Abril 2026